Playing BBR with a userspace network stack

Hajime Tazaki

IIJ Research Laboratory
Tokyo, Japan

1. Motivation

Linux kernel library (LKL), presented at several venues [6][3], is aimed to execute Linux kernel code upon various environments such as Linux userspace, Windows userspace, hypervisors, bootloader, etc. With the userspace deployments, an application can benefit new additional features such as TCP extensions without involving the update of host kernel.

This characteristic of network stack personality is useful since we do not have to instantiate a virtual machine to use a new feature of network stack. Instead, we just need a single (userspace) process, using different network stack apart from the one in host operating system, to introduce new features. Furthermore, this architecture benefits over other userspace network stack due to its maturity of the protocol implementation where other full-scratch or porting based implementations merely have.

One concern of userspace network stack in general, and discussed during the netdev 1.2 conference [1], is the achieved timing accuracy in userspace which the important network feature such as packet pacing and transport protocols require.

In this paper, we report our investigation on how a userspace network stack behaves under the condition which requires highly-accurate timer events on the packet processing.

2. Steps to Improve TCP BBR Goodput

We used the goodput performance measurement of BBR [2], a congestion control algorithm of TCP, as a reference of the measurement. Since BBR involves various features of Linux kernel such as packet pacing with high resolution timer (hrtimer), TCP segmentation offload (TSO), and precise packet timestamping, it is reasonable to see how LKL behaves with various conditions.

The followings is the list of improvement into LKL which we made during this performance study.

- `sched_clock()` implementation in arch/lkl
- add `sysctl` configuration interface for the socket buffer size configuration
- make system memory size configurable
- enable `CONFIG_HIGH_RES_TIMERS`
- add `sch-fq` packet scheduler
- LKL specific scheduler improvements

2.1 `sched_clock()`

Linux kernel has a function, named `sched_clock()`, to provide nanosecond-unit values of current time. By default it returns the tick counter value (i.e., `jiffies`), which the resolution is bound to the tick interval (HZ), as depicted in figure 1. Thus timestamps obtained from this function can describe over than 1/HZ second (e.g., 10 msec in case of HZ=100).

```
unsigned long long __weak
          sched_clock(void)
{
 return (unsigned long long)
   (jiffies - INITIAL_JIFFIES)
     * (NSEC_PER_SEC / HZ);
}
```

Figure 1: sched_clock() function (kernel/sched/clock.c).

This is enough in most case but the measurement of round-trip-time (RTT) used by TCP subsystem requires more precise value if the RTT is very short (e.g., microsecond order).

An architecture in Linux kernel can override the function to provide more precise time value so, the LKL architecture implemented this function in its architecture specific code. The result of this addition makes the RTT measurement precise so that BBR can correctly compute its outgoing window size.

2.2 `sysctl` configuration interface for LKL

When the packet generated from TCP applications goes through a link to the destination, the endpoints need to have a buffer based on its delivery delay and loss ratio. If the buffer at both endpoints are full, it restricts to enqueue new packets until the applications or the other endpoint dequeue the data in buffers. Especially the strength of BBR is seen during packet transmission under the lossy link, which require a proper buffer size configuration of the network stack.

Linux kernel has its interface to configure this buffer size by `sysctl` command (or `proc` filesystem), but LKL did

not have this. An LKL dedicated interface is implemented by an environmental variable `LKL_HIJACK_SYSCTL` (shown in figure 2) to configure the buffer size.

```
LKL_HIJACK_SYSCTL="net.ipv4.tcp_wmem=\
 4096 87380 21474830"
```

Figure 2: sysctl interface.

2.3 Let system memory configurable

Since LKL does not have direct access to a system memory, we need to manually configure the memory size information to an LKL instance during its initialization. It can be changed by each LKL application but it is not configurable if the LKL is used via its library and only the default value (64M bytes) is applied.

The socket buffer size, which we mentioned the previous section, is restricted with available system memory to increase so that a system does not fall into a memory-scare condition.

An environmental variable (shown in figure 3) is also introduced to configure this memory size so that an LKL application used via a library can increase the socket buffer.

```
LKL_HIJACK_BOOT_CMDLINE="mem=100M"
```

Figure 3: boot command line configuration interface.

2.4 Enable `CONFIG_HIGH_RES_TIMERS` configuration

Linux high resolution timer subsystem [4] is the way to invoke timer events which expire in very short duration. If this feature is disabled, the resolution of every timer is bound to tick interval based on HZ configuration, but if enabled it provides nanosecond order resolution if a system is able to do.

In order to use the feature, we just enabled it in a default configuration file (i.e., arch/lkl/Kconfig).

2.5 Add `sch_fq` configuration

The packet pacing feature working with `fq` packet scheduler is an integral component of BBR as the author advertised [2]. In standard Linux, it is enabled by `tc` command to employ this feature onto a NIC, but since LKL does not have an external process at the present moment, we cannot use `tc` command as it is.

```
LKL_HIJACK_NET_QDISC="root|fq"
```

Figure 4: tc configuration interface.

We provided another environmental variable (shown in figure 4) instead to configure the packet scheduler of interests. Note that the current interface is very limited to put parameters (and we are very lazy), we cannot change the default parameters of a scheduler: further configuration should complicate the command line so we may need to have an alternate way such as a system call proxy as LibOS does [7] (reused from rump kernel [5]) to configure this variable.

2.6 LKL scheduler improvement

Although all of the previous described update on LKL, the performance of BBR in a typical network configuration, as described in figure 6, does not achieve as standard Linux does. With the profiling to the software execution on both Linux and LKL environments, we found that the timer event executed by packet pacing were delivered slower than those were scheduled.

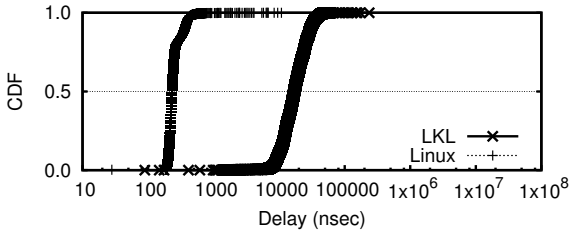


Figure 5: Delivery delay of timer events with high-resolution timer.

The figure 5 illustrates how much delay the timer events scheduled by `fq` packet scheduler is added compared to the (originally) scheduled time. While under standard Linux kernel execution with BBR and `fq`, the event delivery delay are mostly around 100–500 nanoseconds, those went over 10–100 microseconds under LKL . As a result and shows in Table 1, the BBR goodput results of LKL falls into 145.32 Mbps while Linux achieves 8.6 Gbps which is expected result.

Table 1: Goodput performance of TCP with BBR and cubic.

congestion control	Linux (Mbps)	LKL (Mbps)
BBR	8602.40	145.32
cubic	632.63	118.71

Then, we closely looked at the LKL's case on how a timer event is triggered and how much time it takes. Profiling into timer events reveals that a minimum duration for timer interval was about 60 microsecond, which fine-grained timer interval in sub-microsecond, achieved in standard Linux, was not possible.

To alleviate this issue, we changed the scheduler implementation of LKL to avoid a system call invocation (i.e., `clock_nanosleep(2)`) when there are immediate tasks in run queue. The system call is an easy way to block a process in a short minute, but it takes, at least, 40 microsecond[1] which makes the delivery delay longer.

[1] with Linux 4.9.0 kernel

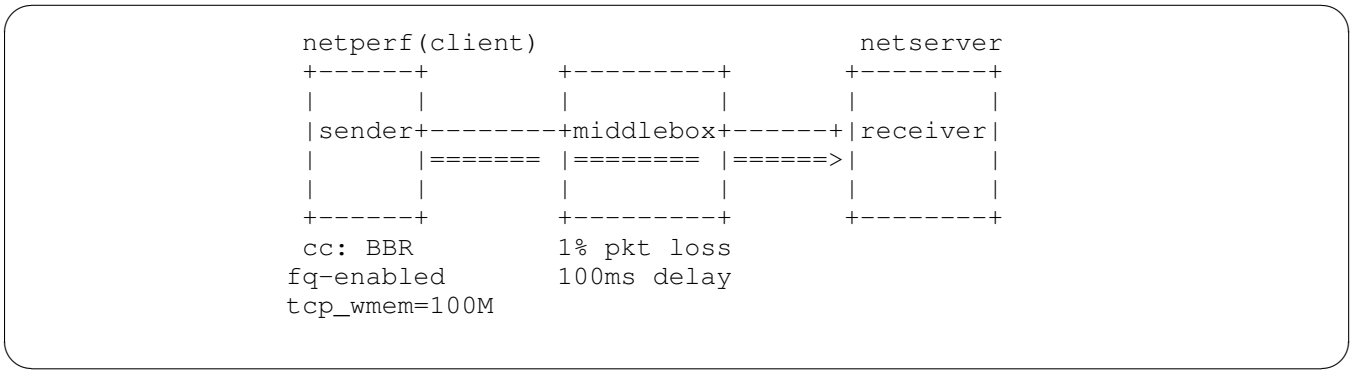


Figure 6: Topology to BBR performance measurement.

```
1  int sleep(u64 nsec) {
2    /* fast path */
3    while (1) {
4     if (nsec < 10*1000) {
5       clock_gettime(CLOCK_MONOTONIC, &now);
6       if (now - start > nsec)
7         return;
8     }
9    }
10
11   /* slow path */
12   return syscall(SYS_clock_nanosleep)
13 }
```

Figure 7: clock_nanosleep(2) update.

The figure 7 is our update to provide a fast path to block a process by busy-polling the clock variable instead of calling system call. The change makes the particular sleep code shorter from 40 microsecond to sub-microsecond.

2.7 Goodput measurement

By adapting all modifications into LKL, and a great contribution of the past LKL development including by Chu et al.[3], the netperf goodput with TCP_STREAM improves a lot. Figure 8 plots the goodput as a function of the socket buffer size on the sender side. Comparing with the result in Table 1, the improvement with LKL is about 28 times (i.e., from 145.32 Mbps to 4.4 Gbps at maximum), which is a huge gain. Note that even if we increased the buffer size to 2G bytes, the LKL goodput was not faster than that of shorter buffer. This indicates this is the upper bound for this configuration with the current implementation, which we need further investigation on.

3. Summary and Further Directions

Network stacks in userspace has the complex situation if the executed code depends on a severe timing. This report has confirmed that 1) this is right and 2) the extensive profiling

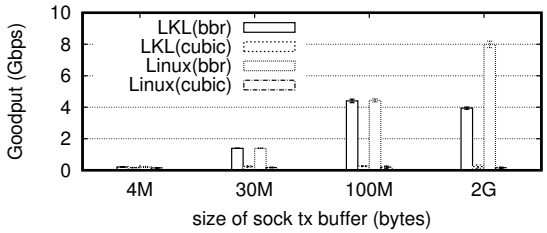


Figure 8: TCP_STREAM goodput by netperf with all of improvements.

can reveal the bottlenecks and suggests the improvement on a particular userspace network stack.

We will investigate more for the degraded performance from the standard Linux execution which was shown in our benchmarks. Additionally more scheduler intensive workload such as under the condition with heavy background tasks shall be examined in order to understand how a userspace network stack can do a critical job in various situations.

References

1. Netdev 1.2, *User Space TCP based on LKL (video, at 34m18s)*, 2017. (Accessed Apr 4th 2017).
2. Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson, *BBR: Congestion-based Congestion Control*, Commun. ACM **60** (January 2017), no. 2, 58–66.
3. HK Jerry Chu and Yuan Liu, *User Space TCP-Getting LKL Ready for the Prime Time*, Linux Netdev 1.2 (October 2016).
4. Thomas Gleixner and Douglas Niehaus, *Hrtimers and Beyond: Transforming the {Linux} Time Subsystems*, Proceedings of the linux symposium, 2006, pp. 333–346.
5. Antti Kantee, *The design and implementation of the anykernel and rump kernels, 2nd edition*, 2016.
6. Octavian Purdila, Lucian Adrian Grijincu, and Nicolae Roedunet International Conference RoEduNet 2010 9th Tapus, *LKL: The Linux kernel library*, Roedunet international conference (roedunet), 2010 9th, 2010, pp. 328–333.
7. Hajime Tazaki, Ryo Nakamura, and Yuji Sekiya, *Library operating system with mainline Linux kernel*, 2015.